

この章では、オブジェクトの宣言について学ぶことで、クラスに関する理解を深めていきます。この知識を活用すれば、プロジェクト全体にわたる行動を効率的に管理できるようになります。

Object declarations

オブジェクトの宣言

`SmartMessenger` クラス内の `receiveMessage()` 関数を書き直す必要はありません。ある意味では、クラスを宣言すると同時に、その唯一のインスタンスも生成することになります。オブジェクト宣言は、プログラムの単一の参照ポイントとして使用するクラスを作成したい場合や、システム全体での動作を調整したい場合に役立ちます。

インスタンスが1つしかなく、そのインスタンスに簡単にアクセスできるクラスを **シングルトン** と呼びます。

{style="tip"}

Kotlin のオブジェクトは **lazy(遅延評価)** であり、アクセスされたときにのみ生成されます。また、Kotlin ではすべてのオブジェクトがスレッドセーフな方法で生成されるよう保証されているため、手動でこれを確認する必要はありません。

オブジェクト宣言を作成するには、`object` キーワードを使用します：

```
object DoAuth {}
```

`object` の名前の後に、中括弧 `{}` で囲まれたオブジェクト本体内に、プロパティやメンバ関数を追加します。

オブジェクトにはコンストラクタがないため、クラスのようなヘッダーファイルはありません。

{style="note"}

たとえば、認証を担当する `DoAuth` というオブジェクトを作成したい場合を考えてみましょう：

```
object DoAuth {
    fun takeParams(username: String, password: String) {
        println("input Auth parameters = $username:$password")
    }
}

fun main(){
    // takeParams() 関数が呼び出されると、オブジェクトが作成されます
    DoAuth.takeParams("coding_ninja", "N1njaC0ding!")
    // input Auth parameters = coding_ninja:N1njaC0ding!
}
```

{kotlin-runnable="true" id="kotlin-tour-object-declarations"}

このオブジェクトには、`takeParams` というメンバー関数があり、この関数は `username` と `password` という変数を引数として受け取り、コンソールに文字列を出力します。The `DoAuth` object is only created when the function is called for the first time.

オブジェクトはクラスやインターフェースから継承することができます。例えば：

```
interface Auth {
    fun takeParams(username: String, password: String)
}

object DoAuth : Auth {
    override fun takeParams(username: String, password: String) {
        println("input Auth parameters = $username:$password")
    }
}
```

{style="note"}

Data objects

データオブジェクト

オブジェクト宣言の内容を印刷しやすくするために、Kotlinには**data**オブジェクトが用意されています。初心者向けツアーで学んだデータクラスと同様に、データオブジェクトには自動的に追加のメンバー関数が備わっています：`toString()` and `equals()`.

データクラスとは異なり、データオブジェクトには `copy()` メンバ関数が自動的に用意されていません。これは、データオブジェクトにはコピーできない単一のインスタンスしか存在しないためです。

{type="note"}

データオブジェクトを作成するには、オブジェクト宣言と同じ構文を使用しますが、その先頭に `data` キーワードを付けます。

```
data object AppConfig {}
```

例えば：

```
data object AppConfig {
    var appName: String = "My Application"
    var version: String = "1.0.0"
}

fun main() {
    println(AppConfig)
    // AppConfig
```

```
println(AppConfig.appName)
// My Application
}
```

```
{kotlin-runnable="true" id="kotlin-tour-data-objects"}
```

データオブジェクトの詳細については、[オブジェクトの宣言と式\(Object declarations and expressions\)](#)を参照してください。

Companion objects

関連オブジェクト(コンパニオン・オブジェクト)

Kotlinでは、クラスに**コンパニオンオブジェクト**というオブジェクトを定義することができます。1つのクラスにつき、**1つ**のコンパニオンオブジェクトしか定義できません。コンパニオンオブジェクトは、そのクラスが初めて参照されたときにのみ作成されます。

コンパニオンオブジェクト内で宣言されたプロパティや関数は、すべてのクラスインスタンス間で共有されます。

クラス内にコンパニオンオブジェクトを作成するには、オブジェクト宣言と同じ構文を使用しますが、その前に **companion** キーワードを付けます。

```
companion object Bonger {}
```

コンパニオンオブジェクトに名前を付ける必要はありません。名前を定義しない場合、デフォルトは **Companion** になります。

```
{style="note"}
```

コンパニオン・オブジェクトのプロパティや関数にアクセスするには、クラス名を指定します。例：

```
class BigBen {
    companion object Bonger {
        fun getBongs(nTimes: Int) {
            repeat(nTimes) { print("BONG ") }
        }
    }
}

fun main() {
    // コンパニオンオブジェクトは、そのクラスが初めて参照されたときに作成されます。
    BigBen.getBongs(12)
    // BONG BONG BONG BONG BONG BONG BONG BONG BONG BONG BONG BONG
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-classes-companion-object"}
```

この例では、**Bonger** というコンパニオンオブジェクトを含む **BigBen** というクラスを作成します。このコンパニオンオブジェクトには、**getBongs()** というメンバ関数があり、整数を受け取り、その整数の値と同じ回数だけ「**BONG**」をコンソールに出力します。

main()関数内では、クラス名を指定して**getBongs()**関数が呼び出されます。この時点でコンパニオンオブジェクトが作成されます。**getBongs()** 関数は、引数 **12** を渡して呼び出されます。

詳細については、[オブジェクトの宣言と式\(Object declarations and expressions\)](#)を参照してください。

演習 {completion-point="true"}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="objects-exercise-1"}

あなたはコーヒーショップを経営しており、顧客の注文を管理するシステムを導入しています。以下のコードを検討し、**main()**関数内の次のコードが正常に実行されるように、2番目のデータオブジェクトの宣言を完成させてください：

|---|---|

```
interface Order {
    val orderId: String
    val customerName: String
    val orderTotal: Double
}

data object OrderOne: Order {
    override val orderId = "001"
    override val customerName = "Alice"
    override val orderTotal = 15.50
}

data object // Write your code here

fun main() {
    // 各データオブジェクトの名前を出力する
    println("Order name: $OrderOne")
    // Order name: OrderOne
    println("Order name: $OrderTwo")
    // Order name: OrderTwo

    // 注文が同一かどうかを確認する
    println("Are the two orders identical? ${OrderOne == OrderTwo}")
    // Are the two orders identical? false

    if (OrderOne == OrderTwo) {
        println("The orders are identical.")
    } else {
        println("The orders are unique.")
        // The orders are unique.
    }

    println("Do the orders have the same customer name? ${OrderOne.customerName ==
```

```
OrderTwo.customerName}")
    // Do the orders have the same customer name? false
}
```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-objects-exercise-1"}
```

```
|---|---|
```

```
interface Order {
    val orderId: String
    val customerName: String
    val orderTotal: Double
}

data object OrderOne: Order {
    override val orderId = "001"
    override val customerName = "Alice"
    override val orderTotal = 15.50
}

data object OrderTwo: Order {
    override val orderId = "002"
    override val customerName = "Bob"
    override val orderTotal = 12.75
}

fun main() {
    // 各データオブジェクトの名前を出力する
    println("Order name: $OrderOne")
    // Order name: OrderOne
    println("Order name: $OrderTwo")
    // Order name: OrderTwo

    // 注文内容が同一かどうかを確認する
    println("Are the two orders identical? ${OrderOne == OrderTwo}")
    // Are the two orders identical? false

    if (OrderOne == OrderTwo) {
        println("The orders are identical.")
    } else {
        println("The orders are unique.")
        // The orders are unique.
    }

    println("Do the orders have the same customer name? ${OrderOne.customerName == OrderTwo.customerName}")
    // Do the orders have the same customer name? false
}
```

```
{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-objects-solution-1"}
```

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="objects-exercise-2"}

Vehicle インターフェースを継承するオブジェクト宣言を作成し、独自の乗り物タイプ **FlyingSkateboard** を作成します。 `main()` 関数内の以下のコードが正常に実行されるように、オブジェクトに `name` プロパティと `move()` 関数を実装してください：

|---|---|

```
interface Vehicle {
    val name: String
    fun move(): String
}

object // Write your code here

fun main() {
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.move()}")
    // Flying Skateboard: Glides through the air with a hover engine
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.fly()}")
    // Flying Skateboard: Wooooooooo
}
```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-objects-exercise-2"}
```

|---|---|

```
interface Vehicle {
    val name: String
    fun move(): String
}

object FlyingSkateboard : Vehicle {
    override val name = "Flying Skateboard"
    override fun move() = "Glides through the air with a hover engine"

    fun fly(): String = "Wooooooooo"
}

fun main() {
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.move()}")
    // Flying Skateboard: Glides through the air with a hover engine
    println("${FlyingSkateboard.name}: ${FlyingSkateboard.fly()}")
    // Flying Skateboard: Wooooooooo
}
```

```
{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-objects-solution-2"}
```

課題 3 {initial-collapse-state="collapsed" collapsible="true" id="objects-exercise-3"}

あなたは、あるアプリ用のユーザー登録モジュールを開発しています。 `User` クラスに関連付けられたメールアドレスの検証は維持したいが、メールアドレスが無効な場合に不必要な `User` インスタンスを作成したくない。

この演習では、`@` と `.` の両方が含まれているメールアドレスを有効なものとしなします。 `main()` 関数内の以下のコードが正常に実行されるように、データクラスを完成させてください：

Add an email validation function in a companion object for the `User` class so that you can call the function directly on `User`.

|---|---|

```
data class User(val name: String, val email: String) {
    // Write your code here
}

fun main() {
    val candidates = listOf(
        Pair("Alice", "alice@example.com"),
        Pair("Bob", "bob2example-com")
    )

    for ((name, email) in candidates) {
        if (User.isValidEmail(email)) {
            val user = User(name, email)
            println("Registered: ${user.name}, ${user.email}")
            // Registered: Alice, alice@example.com
        } else {
            println("Error: '${email}' is not valid. The email should contain '@'
and '.'")
            // Error: 'bob2example-com' is not valid. The email should contain '@'
and '.'
        }
    }
}
```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-objects-exercise-3"}
```

|---|---|

```
data class User(val name: String, val email: String) {
    companion object {
        fun isValidEmail(email: String): Boolean =
            email.contains('@') && email.contains('.')
    }
}
```

```
    }  
}  
  
fun main() {  
    val candidates = listOf(  
        Pair("Alice", "alice@example.com"),  
        Pair("Bob", "bob2example-com")  
    )  
  
    for ((name, email) in candidates) {  
        if (User.isValidEmail(email)) {  
            val user = User(name, email)  
            println("Registered: ${user.name}, ${user.email}")  
            // Registered: Alice, alice@example.com  
        } else {  
            println("Error: '${email}' is not valid. The email should contain '@'  
and '.'")  
            // Error: 'bob2example-com' is not valid. The email should contain '@'  
and '.'  
        }  
    }  
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-objects-solution-3"}

この演習の延長として、コンパニオンオブジェクト内の関数をファクトリメソッドとして使い、クラスのインスタンスを作成してみてください。 For an example and more information about this pattern, see [オブジェクトの宣言と式\(Object declarations and expressions\)](#).

{style="tip"}

- [Previous step](#)
- [Next step](#)